2FA Demo Project Report

Gitika Khira

Project Title:
Two-Factor Authentication (2FA) Demo using Flask, PyOTP, and Email OTP

---

1. Objective

The purpose of this project is to implement a secure Two-Factor Authentication system for web login. Users are required to enter their username and password, and then verify a One-Time Password (OTP) sent to their registered email before they can log in successfully. This adds an additional layer of security over traditional login systems.

---

2. Technologies Used

Technology	Purpose
Python 3.x	Core programming language
Flask	Web framework to handle routing, sessions, and templates
pyotp	Generate Time-Based One-Time Passwords (TOTP)
bcrypt	Password hashing for security
smtplib, email.mime	Sending OTP emails to users
HTML, Jinja2	Frontend templates (login, register, OTP verification, success)
CSS	Styling the frontend pages
Session	Track user login and OTP attempts

---

3. Features

1. **User Registration:**
    - Users can register with username, password, and email.
    - Passwords are securely hashed using `bcrypt`.

2. **Login with Password Verification:**
    - Username and password authentication against stored hashed credentials.
3. **Two-Factor Authentication via Email OTP:**
    - OTP generated using TOTP (PyOTP).
    - OTP expires after 2 minutes.
    - OTP sent via email to the registered address.
4. **OTP Verification:**
    - Users enter OTP on a verification page.
    - Maximum 3 attempts allowed per session.
    - Expired OTP or exceeding attempts redirects the user to login.
5. **Session Management:**
    - Tracks logged-in users and OTP attempts.
    - Clears session after successful login or expiration.
6. **Flash Messages:**
    - Provides real-time feedback: registration success, login errors, OTP messages, and verification status.

---

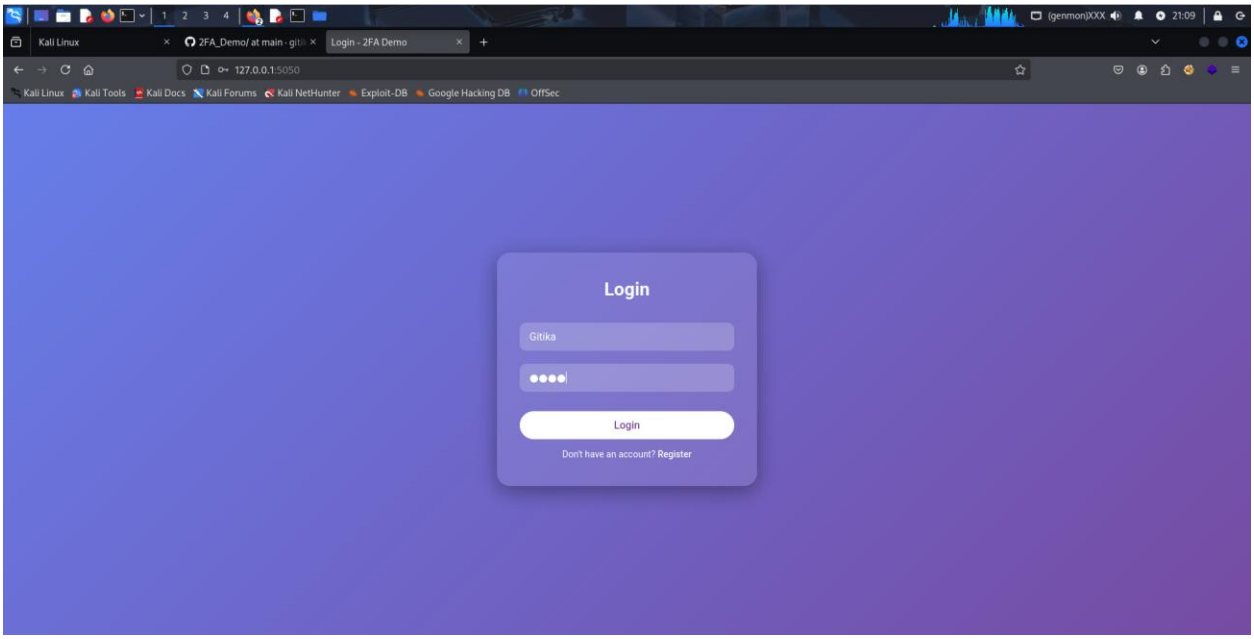
4. Application Flow

1. **Registration:** `/register`
    - User provides username, password, and email.
    - Password hashed and stored in memory.
    - Redirected to login page.
2. **Login:** `/login`
    - User enters username and password.
    - Password verified; OTP generated using PyOTP.
    - OTP emailed to the user.
    - Redirected to OTP verification page.
3. **OTP Verification:** `/two_factor`
    - User enters OTP.
    - System checks OTP validity and attempt count.
    - On success, user redirected to success page.
    - On failure or expiration, redirected to login page.
4. **Success Page:** `/success.html`
    - Displays login success message and username.
    - Logout option provided to return to login page.

5. Frontend Implementation

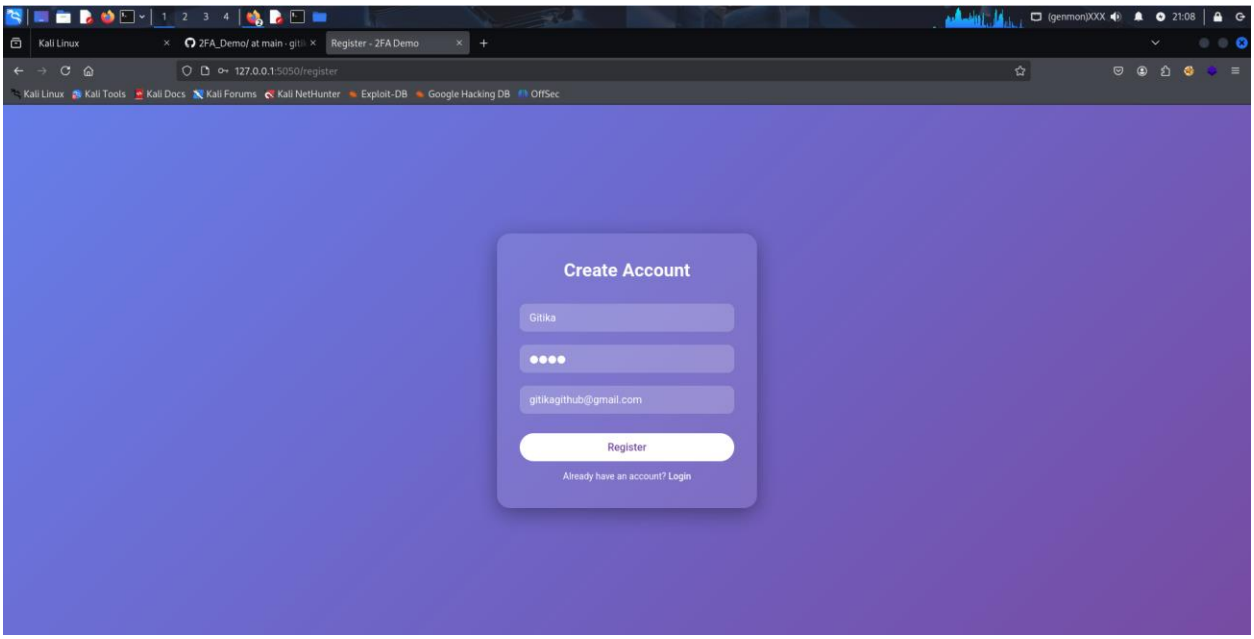
login.html

- Provides username/password input.
- Displays flash messages for errors.
- Link to registration page.



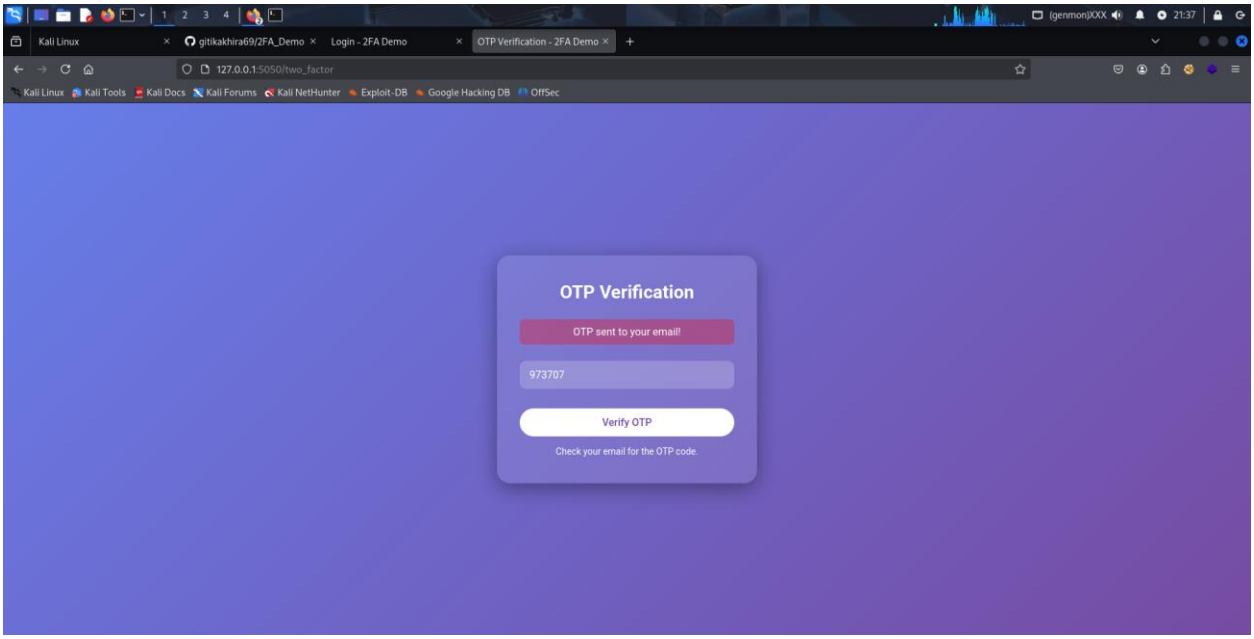
register.html

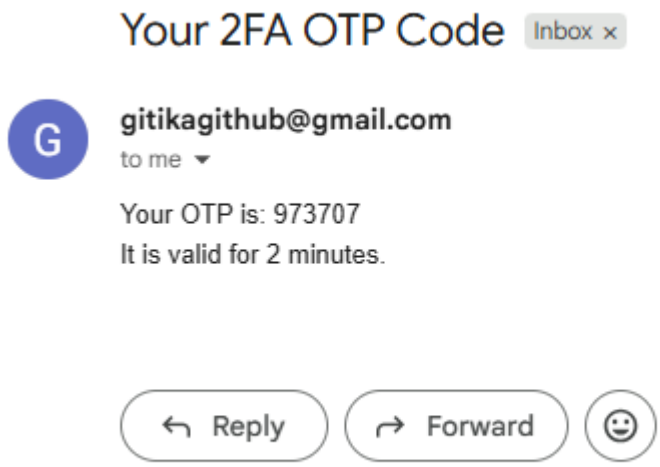
- Allows new users to register.
- Validates username uniqueness.
- Displays success or error flash messages.
- Link to login page.



otp.html

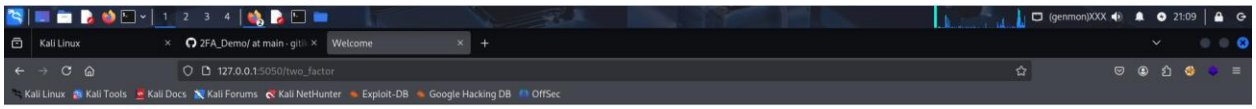
- Input for entering OTP.
- Displays flash messages for expired/invalid OTP.
- Instruction to check email for OTP.





success.html

- Confirms successful login.
- Displays username and “2FA verified” message.
- Logout button to return to login page.



CSS (style.css)

- Glassmorphism container design with blurred backgrounds.
- Gradient background for login/register/OTP pages.
- Stylish input fields, buttons, and flash message styling.
- Fonts imported from Google Fonts (Roboto).

---

6. Security Features

- **Password Hashing:** Uses bcrypt to prevent storing plain passwords.
- **OTP Expiry:** OTP valid only for 2 minutes.
- **Attempt Limiting:** Maximum of 3 OTP attempts.
- **Session Management:** Restricts access to OTP verification without login.

---

7. Code Structure

```
app.py              # Main Flask application
templates/
    ├─ login.html       # Login page
    ├─ register.html    # Registration page
    ├─ otp.html         # OTP verification page
    └─ success.html     # Successful login page
static/
    ├─ style.css        # CSS styling for all pages
    └─ stars.jpg        # Optional background image for success page
```

---

8. How to Run

1. Install dependencies:

```
pip install flask pyotp bcrypt
```

2. Update the email credentials in `app.py`:

```
SENDER_EMAIL = 'your_email@gmail.com'
SENDER_PASSWORD = 'your_app_password'
```

3. Run the Flask server:

```
python app.py
```

4. Open browser at `http://localhost:5050/`

5. Register → Login → Enter OTP → Success.

---

9. Limitations

- OTP stored in memory; server restart loses OTPs.
- SMTP requires Gmail App Password.
- Not suitable for production; lacks database, HTTPS, rate limiting, and logging.

---

10. Conclusion

This 2FA Demo project demonstrates the implementation of a secure login system with email-based OTP verification. It strengthens authentication and can be extended with database storage, SMS OTP, authenticator apps, and production-ready security practices.